HELLA company 업무관리사이트 — 추가 질문 답변서 (에이전트 세팅용)

이 문서는 아래 질문에 대해 **현재까지 확정된 실제 기획 문서와 설계 문서 기준으로** 답변한 것이다.

목적: - AI 에이전트가 환각 없이 프로젝트를 이해하도록 기준점 제공 - SSOT 성격의 핵심 답변서 역할 수행 - AGENTS.md / clone-spec.md / SKILL.md / backend-states.md 작성의 직접 입력값으로 사용

---

1. 프로젝트 개요 및 기술 스택 (AGENTS.md 헌법 제정용)

1-1. 이 프로젝트는 무엇인가?

HELLA company 업무관리사이트는 회사 내부 직원만 사용하는 **업무 전용 통합 운영 웹앱**이다.

이 시스템은 단순한 사내 포털이 아니다. 다음 업무 흐름을 하나의 웹앱에서 통합 운영하는 현장 서비스 운영 시스템이다.

- 로그인 및 권한 분기
- 관리자 대시보드
- 사업분야/작업 카테고리 관리
- 업무 메뉴얼 관리
- 일반 현장 일정 생성/배정/상태관리
- 사원 일정 조회
- 시작/종료/특이사항/비포·애프터 사진 업로드
- 정기청소 참여 기록 및 통계
- NFC 기반 장비 반출/반입
- 미반입 장비 추적
- 감사 로그 / 다운로드 로그 / 설정

즉, 이 프로젝트는 **현장 일정 + 현장 실행 + 사진 증빙 + 사원 기록 + 장비 흐름 + 관리자 통제**를 한 시스템으로 묶는 구조다.

1-2. 기술 스택은 무엇인가?

프론트엔드

- React 18
- Vite
- TypeScript
- React Router
- Tailwind CSS
- Zustand

- TanStack Query

백엔드 / 인프라

- Firebase Auth
- Firestore
- Firebase Storage
- Firebase Functions
- Firebase Hosting

사용 환경

- 관리자: 데스크탑 우선
- 사원: 모바일 우선
- 전체: 반응형 웹앱
- NFC: 안드로이드 Chrome 계열 Web NFC 기준
- 배포: HTTPS 필수

코드 아키텍처 원칙

- 함수형 컴포넌트 사용
- TypeScript 우선
- page 컴포넌트에서 Firebase SDK 직접 호출 금지
- repository / service / function 계층 분리
- UI와 비즈니스 로직 분리

1-3. 에이전트가 절대 건드리면 안 되는 하드 제약은 무엇인가?

다음은 **절대 규칙**이다.

문서 의미를 임의로 바꾸면 안 되는 기준 문서

- docs/01_mvp_spec.md
- docs/02_menu_function_specs.md
- docs/03_project_roadmap.md
- docs/06_permission_policy.md
- docs/07_firestore_rules.md
- docs/08_nfc_engine.md

데이터/보안 구조에서 절대 바꾸면 안 되는 것

- Firestore 컬렉션 이름 변경 금지
- Firestore 핵심 필드명 무단 변경 금지
- Firestore 인덱스 무단 변경/삭제 금지
- Firestore Rules 완화 금지
- Storage Rules 공개형 완화 금지
- audit_logs 수정/삭제 허용 금지
- equipment_logs 클라이언트 직접 쓰기 허용 금지
- role / isActive / currentHolder / status 필드 클라이언트 자유 수정 허용 금지

상태 전이 관련 절대 금지

- schedules.status를 클라이언트 직접 set하는 구조로 변경 금지
- equipments.status를 클라이언트 직접 set하는 구조로 변경 금지
- schedule_workers.workStatus를 검증 없이 직접 수정 가능하게 만들지 말 것
- 이미 제출된 마감 기록을 사원 권한으로 다시 수정 가능하게 만들지 말 것

인증/권한 절대 금지

- 로그인 후 권한 분기 규칙 변경 금지
- 비활성 계정 로그인 허용 금지
- EMPLOYEE가 관리자 라우트에 접근 가능하도록 우회 금지
- ADMIN/SUPER_ADMIN 전용 메뉴를 공통 공개 메뉴로 바꾸지 말 것

NFC / 장비 절대 금지

- 반출/반입을 단순 UI 토글 처리로 바꾸지 말 것
- NFC 처리에서 서버 재검증 생략 금지
- 장비 상태 변경과 로그 생성을 분리 커밋하지 말 것
- 동일 장비 동시 반출 충돌 방어 제거 금지

운영 범위 절대 금지

- 정산/결제 기능 추가 금지
- 외부 고객용 기능 추가 금지
- 회원가입 / 비밀번호 찾기 / 공개 접근 기능 임의 추가 금지

1-4. 에이전트의 언어/행동 규칙은 무엇인가?

- 모든 설명은 한국어로 작성한다.
- 모든 주석은 한국어 우선으로 작성한다.
- 모든 커밋 메시지는 한국어로 작성한다.
- 추측으로 스키마를 바꾸지 않는다.
- 문서 기준 없이 기능을 추가하지 않는다.
- 작업은 초원자 단위로 쪼갠다.
- 한 번에 여러 도메인을 동시에 건드리지 않는다.
- 수정 전 영향 범위를 먼저 적는다.
- 수정 후 검증 항목을 반드시 적는다.
- 보안 규칙을 완화하는 방향으로 문제를 해결하지 않는다.
- 로그/감사 추적을 없애는 방향으로 리팩토링하지 않는다.
- UI polish보다 상태/권한/로그/트랜잭션을 우선한다.

---

2. 브랜드 이식 및 UI/UX 구조 (clone-spec.md 작성용)

2-1. 기존 앱을 HELLA 브랜드로 복제하는가, 아니면 백지에서 새로 만드는가?

이 프로젝트는 **기존 소비자 앱을 스킨갈이하는 구조가 아니다.**

정확히 말하면, - 기존 문서/설계 자산은 이미 많이 준비되어 있고 - 그 자산을 기준으로 - **HELLA company 업무관리 사이트라는 내부 직원용 운영 포털을 새로 구현하는 단계**다.

즉, - 완전 백지 상태는 아님 - 그러나 기존 B2C 앱을 단순 복제하는 것도 아님 - **설계 완료된 운영 시스템을 실제 제품으로 구현하는 단계**라고 보는 것이 맞다.

2-2. 주요 메뉴 카테고리는 어떻게 구성되는가?

관리자 기준 상위 메뉴는 아래 10개다.

```
[
 '대시보드',
 '사업분야별 관리메뉴',
 '업무메뉴별 메뉴얼',
 '정기청소 관리',
 '회사 일정관리',
 '스케줄별 일정표',
 '스케줄 마감',
 '사원관리',
 'NFC 장비 반출·반입관리',
 '설정'
]
```

사원 하단 탭 메뉴는 아래 5개다.

```
[
 '홈',
 '일정',
 '장비',
 '메뉴얼',
 '내정보'
]
```

사원 홈 Quick Action은 아래 4개다.

```
[
 '작업 시작',
 '작업 종료',
 '장비 반출',
 '장비 반입'
]
```

관리자 Quick Action은 아래 4개다.

```
[
 '일정 생성',
 '사원 등록',
 '장비 등록',
 '메뉴얼 등록'
]
```

2-3. HELLA 브랜드 메인 컬러와 디자인 토큰은 무엇인가?

앱 식별자

- 정식 표시명: HELLA company 업무관리사이트
- 보조 설명: 내부 직원 전용 운영 시스템
- 짧은 표기: HELLA OPS

라우트 구조

- 공통 진입: `/login`
- 관리자 앱: `/app/*`
- 사원 앱: `/m/*`

역할별 기본 진입 경로

- EMPLOYEE → `/m/home`
- LEADER → `/m/home`
- ADMIN → `/app/dashboard`
- SUPER_ADMIN → `/app/dashboard`

브랜드 컬러

Primary

- primary-900: `#0F172A`
- primary-800: `#1E293B`
- primary-700: `#334155`
- primary-600: `#475569`
- primary-500: `#64748B`

Brand Accent

- brand-main: `#0F766E`
- brand-main-hover: `#115E59`
- brand-soft: `#CCFBF1`
- brand-border: `#99F6E4`

Background

- bg-app: `#F8FAFC`

- bg-surface: `#FFFFFF`
- bg-muted: `#F1F5F9`
- bg-sidebar: `#0F172A`

Text

- text-primary: `#0F172A`
- text-secondary: `#475569`
- text-muted: `#64748B`
- text-on-dark: `#FFFFFF`

일정 상태 색상

- 예정: `#2563EB`
- 진행중: `#16A34A`
- 완료: `#475569`
- 지연: `#DC2626`
- 취소: `#94A3B8`

장비 상태 색상

- AVAILABLE: `#16A34A`
- CHECKED_OUT: `#2563EB`
- OVERDUE: `#DC2626`
- DISABLED: `#6B7280`

로고 경로 정책

현재 문서상 로고 **실제 파일 경로는 아직 확정 문서로 주어지지 않았다.** 따라서 지금 단계의 안전한 기준은 아래와 같다.

- 로그인 로고 값: `settings.loginLogoUrl`
- 헤더 로고 값: `settings.headerLogoUrl`
- 로고 없을 경우: 텍스트 로고 fallback

즉, **실물 로고 경로를 하드코딩하지 않고 settings 기반 동적 주입**이 현재 설계 기준에 맞다.

2-4. 화면 레이아웃 순서는 어떻게 정의되는가?

로그인 화면 순서

1. 로고
2. 시스템명
3. 안내문
4. 사원 ID 입력
5. 비밀번호 입력
6. 오류 메시지
7. 로그인 버튼
8. 하단 보조 문구

관리자 대시보드 순서

1. KPI Section
2. Alert Section
3. Quick Action Section
4. Chart Section
5. Today Schedule Section
6. Equipment Section
7. Activity Section

사원 홈 순서

1. Header
2. Today Summary Card
3. Quick Action Grid
4. Today Schedule List
5. Equipment Status

사원 일정 화면 순서

1. MobileHeader
2. DateScopeTabs
3. MiniCalendarSection
4. ScheduleFilterBar
5. ScheduleListSection
6. ScheduleDetailBottomSheet

사원 작업 시작/종료/마감 화면 순서

1. MobileHeader
2. ScheduleSummaryCard
3. WorkStatusStepper
4. StartSection
5. EndSection
6. NotesSection
7. BeforePhotoSection
8. AfterPhotoSection
9. ValidationAlertArea
10. BottomActionBar

관리자 장비관리 화면 순서

1. PageHeader
2. ActionBar
3. FilterPanel
4. EquipmentTable
5. EquipmentDetailDrawer
6. AlertPanel

---

3. 초원자 단위(Ultra-Atomic) 기능 명세 (SKILL.md 및 Workflow 설계용)

3-1. 가장 먼저 구현해야 할 핵심 기능 흐름은 무엇인가?

이 프로젝트는 아래 순서가 가장 안전하다.

1단계: 시스템 기반

1. 프로젝트 골격 생성
2. 라우팅 및 권한 분기 골격
3. 로그인 화면 구현
4. Firebase Auth + users 프로필 조회 연결
5. 관리자/사원 공통 레이아웃 분리

2단계: 운영 핵심 화면

1. 관리자 대시보드 1차 구현
2. 사원 홈 1차 구현
3. 관리자 일정관리 목록/캘린더 화면
4. 일정 생성/수정 폼 구현
5. 일정 상세 화면 구현
6. 사원 일정 화면 구현
7. 사원 시작/종료/마감 화면 구현

3단계: 장비 흐름

1. 관리자 장비관리 화면 구현
2. 사원 반출/반입 화면 구현
3. Firestore Rules 1차 적용
4. Functions: 일정 생성 + 배정
5. Functions: 작업 시작/종료/마감
6. Functions: NFC 반출/반입 트랜잭션
7. Storage Rules + 사진 업로드 연동
8. 운영 안정화 테스트

즉, 첫 기능은 **로그인 화면 UI**가 아니라, 더 정확히는 **프로젝트 골격 → 권한 분기 → 로그인 → 프로필 조회 → 역할별 진입** 순서다.

3-2. 15분 단위로 쪼개기 위한 실제 초원자 Step 예시는 무엇인가?

아래는 바로 사용할 수 있는 기준 Step이다.

STEP 1 — 프로젝트 골격 생성

- 목표: React + Vite + TypeScript + Tailwind + Firebase 기본 골격 생성
- 검증: `npm run build` 에러 0개

STEP 2 — 라우팅/권한 분기 골격

- 목표: `/login`, `/app/dashboard`, `/m/home` 기본 라우팅 생성
- 검증: 비로그인 시 보호 라우트 접근 차단

STEP 3 — 로그인 화면 UI

- 목표: 로그인 와이어프레임 그대로 구현
- 검증: 필드/버튼/오류 메시지/모바일 레이아웃 정상

STEP 4 — Auth 연결

- 목표: Firebase Auth + users 프로필 조회 + 역할별 리다이렉트
- 검증: EMPLOYEE는 `/m/home`, ADMIN은 `/app/dashboard`

STEP 5 — 공통 레이아웃

- 목표: AdminLayout / EmployeeLayout 분리
- 검증: 관리자와 사원 셸 UI 분리 성공

STEP 6 — 관리자 대시보드 1차

- 목표: KPI / Alert / Quick Action
- 검증: KPI 카드 4개와 경고 카드 렌더링

STEP 7 — 사원 홈 1차

- 목표: Today Summary / Quick Action / Today Schedule / Equipment Status
- 검증: 모바일 업무 시작 허브 구조 성립

STEP 8 — 관리자 일정관리 목록/캘린더

- 목표: 일정 목록과 캘린더 동시 렌더링
- 검증: 필터/날짜 클릭 동작

STEP 9 — 일정 생성/수정 폼

- 목표: 제목/날짜/시간/사원배정/특이사항 입력 구현
- 검증: 필수값/시간 역전 검증

STEP 10 — 일정 상세 화면

- 목표: 요약/배정사원/마감정보/사진 탭 렌더링
- 검증: 상세 운영 정보 조회 가능

STEP 11 — 사원 일정 화면

- 목표: 본인 일정만 목록 + 바텀시트 상세
- 검증: 시작/마감 CTA 규칙 정상

STEP 12 — 사원 시작/종료/마감 화면

- 목표: 시작 → 종료 → 사진 → 마감 제출 흐름 구현
- 검증: 시작 없이 종료 차단

STEP 13 — 관리자 장비관리

- 목표: 장비 목록/상세/미반입 강조
- 검증: 상태 색상 + 상세 Drawer 정상

STEP 14 — 사원 반출/반입

- 목표: NFC 모드별 화면
- 검증: 반출/반입 모드 혼선 없음

STEP 15 — Firestore Rules 1차

- 목표: 본인/관리자 권한 분리
- 검증: equipment_logs 직접 쓰기 차단

STEP 16 — Functions: 일정 생성 + 배정

- 목표: schedule + schedule_workers 원자 처리
- 검증: 생성 후 worker 문서 수 일치

STEP 17 — Functions: 작업 시작/종료/마감

- 목표: 서버에서 상태 전이 확정
- 검증: 종료 > 시작 검증, 마감 제출 후 동기화

STEP 18 — Functions: NFC 반출/반입

- 목표: 장비 상태 + 로그 동시 트랜잭션
- 검증: AVAILABLE만 반출, CHECKED_OUT/OVERDUE만 반입

STEP 19 — Storage Rules + 사진 업로드

- 목표: 사진 메타와 Storage 권한 일치
- 검증: 업로더/관리자만 접근 가능

STEP 20 — 운영 시나리오 검증

- 목표: 로그인→일정→시작→종료→사진→마감→장비 흐름 전체 점검
- 검증: 컴파일 에러 0 / 권한 우회 0 / 로그 누락 0

3-3. PASS / FAIL 기준의 핵심 원칙은 무엇인가?

공통 PASS 기준

- 컴파일 에러 0개
- 화면이 문서 정의 순서대로 렌더링됨
- 권한 우회 없음
- 본인 데이터/관리자 데이터 경계 명확
- 상태 전이가 문서 기준과 일치함
- 로그 누락 없음

공통 FAIL 기준

- null 접근으로 화면 깨짐
- 비활성 계정 로그인 통과
- 타인 일정 접근 가능
- equipment_logs 직접 생성 가능
- 시작 없이 종료 가능
- 장비 상태와 로그 불일치
- 모바일 레이아웃 깨짐

FAIL 시 대처 원칙

- 조건부 렌더링 강화
- 서버 검증 재강화
- helper 분리
- 상태 전이 검증 재작성
- mock 데이터가 실제 분기를 가리는지 점검

---

4. 데이터베이스 및 백엔드 예외 상태 (Backend States 정의용)

4-1. DB 권한 구조는 어떻게 되는가?

역할은 4개다. - EMPLOYEE - LEADER - ADMIN - SUPER_ADMIN

EMPLOYEE

가능: - 로그인 - 본인 스케줄 조회 - 시작시간 입력 - 종료시간 입력 - 특이사항 작성 - 사진 업로드 - 본인 장비 반출/반입 - 본인 보유 장비 조회 - 메뉴얼 조회

제한: - 사원 목록 수정 불가 - 장비 등록 불가 - 권한 변경 불가 - 전체 이력 엑셀 다운로드 불가

LEADER

가능: - 일반 사원 기능 전체 - 팀원 일정 배정 확인 - 일부 일정 보정 - 현장 기록 검토 - 제한된 집계 조회

ADMIN

가능: - 전 메뉴 접근 - 사업분야 생성/수정/비활성 - 메뉴얼 등록/수정 - 일정 등록/수정/삭제 - 스케줄 배정 - 정기청소 장소/투입인원 관리 - 사원 등록/수정/권한관리 - NFC 장비 등록/매핑/반출반입 이력 조회 - 엑셀 다운로드 - 감사 로그 조회

SUPER_ADMIN

가능: - 관리자 계정 관리 - 시스템 설정 수정 - 데이터 복구성 작업

4-2. Firestore 컬렉션 기준 핵심 구조는 무엇인가?

주요 컬렉션은 아래와 같다.

- `users`
- `business_categories`
- `business_subcategories`
- `manuals`
- `schedules`
- `schedule_workers`
- `schedule_closings`
- `schedule_photos`
- `regular_cleaning_sites`
- `regular_cleaning_attendance`
- `equipments`
- `equipment_logs`
- `nfc_tag_mappings`
- `audit_logs`
- `download_logs`
- `settings`

권한 설계 핵심은 다음이다.

- `users`: 본인 또는 ADMIN+
- `schedules`: ADMIN 전체 / 사원은 본인 배정 일정만
- `schedule_workers`: 본인 시작/종료만 제한 수정
- `schedule_closings`: 본인 배정 일정만 생성, 수정은 관리자 중심
- `schedule_photos`: 업로더 또는 관리자
- `equipments`: 읽기 로그인 사용자 / 쓰기 ADMIN
- `equipment_logs`: 클라이언트 쓰기 금지, Functions 전용
- `audit_logs`: 수정/삭제 금지
- `settings`: ADMIN 또는 SUPER_ADMIN 중심

4-3. Boring States(예외 상태)는 어떻게 처리해야 하는가?

이 프로젝트는 **에러를 터뜨리기보다 우아하게 비어 있는 상태를 보여주는 것**이 중요하다. 다만, 권한/보안/상태 위반은 반드시 차단해야 한다.

로그인 화면

- 로고 없음 → 텍스트 로고 fallback
- companyName 없음 → 고정명 `HELLA company 업무관리사이트`
- 인증 실패 → 일반화된 오류 문구
- 비활성 계정 → 차단 메시지

관리자 대시보드

- KPI 데이터 0건 → 0으로 표시

- 오늘 일정 없음 → empty widget
- alert 0건 → “현재 즉시 처리할 항목이 없습니다”
- 일부 위젯 실패 → 위젯 단위 error card

사원 홈

- 오늘 일정 0건 → “오늘 일정 없음” 카드
- 장비 0건 → “현재 보유 장비 없음”
- 최근 작업 없음 → “최근 작업 기록 없음”

관리자 일정관리

- 필터 결과 0건 → “조건에 맞는 일정이 없습니다”
- category/subcategory 미로드 → selector disabled + retry
- 일정 0건이어도 캘린더/필터 UI는 유지

사원 일정 화면

- 특정 날짜 일정 0건 → EmptyStateSection 유지
- 전체 배정 일정 0건 → “배정된 일정이 없습니다”
- 본인 배정 아닌 scheduleId 접근 → 차단 메시지

사원 시작/종료/마감 화면

- 시작 없이 종료 시도 → blocking error
- 종료 < 시작 → blocking error
- 비포/애프터 0장 → 경고 또는 정책상 차단
- 업로드 실패 사진 있음 → 제출 차단
- 제출 완료 후 재진입 → 읽기 전용

장비 반출/반입

- Web NFC 미지원 → 명확한 안내 + fallback 정책 표시
- 태그 미등록 → 처리 차단
- 이미 반출중 장비 재반출 시도 → 현재 보유자/반출 시각 표시
- 이미 보관 상태 장비 반입 시도 → 차단 메시지
- 서버 지연 → 로딩 + 중복 클릭 차단

관리자 장비관리

- 장비 없음 → “등록된 장비가 없습니다”
- 로그 없음 → “이력이 없습니다”
- overdue 없음 → 정상 상태 안내
- 중복 ACTIVE 매핑 감지 → 관리자 오류 배너

4-4. 신규 계정이라 할당된 업무가 하나도 없을 때는 무엇을 보여줘야 하는가?

이 경우는 에러가 아니라 **정상적인 empty state**로 처리해야 한다.

사원 홈

- 메인 문구: `오늘 일정 없음`
- 보조 문구: `현재 배정된 업무가 없습니다. 새로운 일정이 등록되면 이 화면에 표시됩니다.`

사원 일정 화면

- 메인 문구: `배정된 일정이 없습니다`
- 보조 문구: `다른 날짜를 선택하거나, 관리자에게 일정 배정 여부를 확인해 주세요.`

장비 영역

- 메인 문구: `현재 보유 장비 없음`
- 보조 문구: `반출 처리된 장비가 있으면 이곳에 표시됩니다.`

핵심은 **화면을 통째로 숨기지 않고, 사용자가 다음 행동을 이해할 수 있게 안내 문구를 남기는 것**이다.

---

5. 최종 정리

위 질문들에 대한 가장 정확한 답은 다음과 같이 요약된다.

1. 이 프로젝트는 **React + Firebase 기반 내부 직원용 통합 운영 웹앱**이다.
2. 현재는 **아이디어 단계가 아니라 설계 완료 후 구현 직전 단계**다.
3. 주요 메뉴는 대시보드, 사업분야, 메뉴얼, 정기청소, 일정관리, 일정표, 마감, 사원관리, NFC 장비, 설정으로 고정돼 있다.
4. 브랜드 토큰과 라우트 구조도 이미 정리돼 있으며, 로고는 settings 기반 동적 주입이 가장 안전하다.
5. 구현 순서는 로그인/권한/레이아웃 → 관리자/사원 핵심 화면 → Rules → Functions → NFC/사진/안정화 순서가 맞다.
6. DB 권한은 EMPLOYEE / LEADER / ADMIN / SUPER_ADMIN 4단계다.
7. boring states는 에러보다 **명확한 empty state + graceful fallback** 전략으로 처리해야 한다.
8. 이 시스템의 핵심은 **상태를 클라이언트가 결정하지 않고 서버가 결정한다**는 점이다.

이 문서는 바로 다음 단계에서 아래 파일로 분리해도 된다. - AGENTS.md - clone-spec.md - SKILL.md - backend-states.md